Lesson 1 — How The Main Window Lives

Purpose

This lesson is the first guided reading of the existing WPF application.

Its purpose is not to teach the whole project at once. Its purpose is to let you see how a desktop application becomes a living object: something starts, a window appears, data is exposed, the interface reacts, the user acts, and code responds.

The material for this lesson is:

- `project/src/TrajectoryPlanner.App/App.xaml`
- `project/src/TrajectoryPlanner.App/MainWindow.xaml`
- `project/src/TrajectoryPlanner.App/MainWindow.xaml.cs`
- `project/src/TrajectoryPlanner.App/PrototypeModels.cs`

What You Should Learn Here

At the end of this lesson, you should be able to explain:

1. how the application starts;
2. how WPF knows which window to open;
3. what XAML does and what C# code does;
4. why `{Binding ...}` exists;
5. what `DataContext` means in this app;
6. why `INotifyPropertyChanged` is necessary;
7. why `ObservableCollection<T>` is used;
8. what happens when the user clicks `Compute`.

The Right Attitude

Do not ask, at the beginning, whether this app is architecturally perfect.

Ask a more useful question:

> What must exist for this window to live and react?

That question is more fertile. It leads us toward understanding before judgment.

Later we will judge the design. First we must learn to see it.

The Four Pieces

This lesson studies four pieces that form one vertical slice.

Piece 1 — `App.xaml`

This file is the application declaration.

In the current project it is extremely small, and that is good for us. It says, in substance:

- this is the application class;
- these are the XAML namespaces in use;
- the startup window is `MainWindow.xaml`.

The key line is the startup declaration. It tells WPF which window to create first.

So the first simple mental model is:

`App.xaml` does not contain the application logic. It declares the application entry configuration for WPF.

Piece 2 — `MainWindow.xaml`

This file declares the visible structure of the main window.

It is not the computation engine. It is the declarative description of:

- controls;
- layout;
- bindings;
- visual grouping;
- event hookup points.

Here you already meet one of the deepest ideas of WPF:

> structure can be declared separately from behavior.

In practice:

- XAML says what the window contains;
- C# says what the window does.

This separation is not absolute, but it is the first strong conceptual border you should learn.

Piece 3 — `MainWindow.xaml.cs`

This is the code-behind of the window.

Here the window becomes an active C# object. The class:

- initializes the window;
- sets `DataContext = this`;
- stores state in properties and collections;
- reacts to button clicks;
- calls the prototype trajectory service;
- refreshes the charts and report rows.

This means the window is not only a visual surface. It is also a stateful object participating in the application logic.

For learning purposes, this is extremely useful because it keeps the interaction loop visible.

Piece 4 — `PrototypeModels.cs`

This file contains small data-oriented classes and records used by the app layer.

It is important because it shows that not every class in an application is a “big service” or a “main object”. Some classes exist simply to carry data or expose bindable state cleanly.

This file also introduces the real need for `INotifyPropertyChanged`, because some objects here must inform the UI when a property changes.

The First Mental Model

Before going into details, hold this complete mental chain:

1. WPF starts the application.
2. `App.xaml` points to `MainWindow.xaml`.
3. `MainWindow.xaml` describes the window.
4. `MainWindow.xaml.cs` gives the window behavior.
5. The window exposes data through properties and collections.
6. XAML binds controls to those properties and collections.
7. User actions trigger event handlers.
8. Event handlers call computation services.
9. Results are written back into bound state.
10. The UI updates.

This is the basic circulation of life in the current app.

XAML Versus C

One of the first confusions in WPF is the relation between XAML and C#.

The cleanest initial distinction is this:

- XAML is mainly declarative;
- C# is mainly behavioral.

Examples from the current app:

- buttons, grids, tab controls, text blocks, data grids, and plots are declared in XAML;
- what happens when `Add Segment`, `Remove Segment`, or `Compute` is clicked is written in C#;
- the XAML says a `TextBlock` should show `{Binding StatusMessage}`;
- the C# code changes `StatusMessage`.

So XAML is not "magic markup". It is a way to declare an object graph that WPF builds for the window.

What `DataContext = this` Means Here

In the window constructor, the code sets:

```
DataContext = this;
```

For this app, that means:

> when XAML asks for a binding like `{Binding StatusMessage}`, it will look for `StatusMessage` on the `MainWindow` object itself.

This is one of the most important lines in the whole first lesson.

Why? Because it explains why the XAML can "see" properties such as:

- `StatusMessage`
- `Segments`
- `SelectedSegment`
- `CanRemoveSegment`
- `ExtremaReportRows`
- `CoefficientReportRows`
- `CursorReadoutRows`

Without `DataContext`, those bindings would not know where to look.

Why `{Binding ...}` Exists

Binding is WPF's mechanism for connecting UI elements to data.

In this app:

- a `TextBlock` binds to `StatusMessage`;
- a `TabControl` binds to `Segments`;
- the selected tab binds to `SelectedSegment`;
- button enabled state binds to `CanRemoveSegment`;
- data grids bind to report collections.

So binding is not decoration. Binding is the wire between the visible interface and the object's state.

Why `INotifyPropertyChanged` Exists

Suppose the UI is bound to `StatusMessage`.

If the C# code changes the property value, how would the UI know it must redraw that text?

That is the job of `INotifyPropertyChanged`.

When a property changes, the object raises a notification. WPF listens for that notification and refreshes the bound element.

So `INotifyPropertyChanged` is not a formal burden invented by the framework. It is the protocol that lets the object say:

> A value you care about has changed. Update yourself.

In this lesson, you should connect the idea of *property* with the idea of *change notification*. In WPF, these often travel together.

Why `ObservableCollection<T>` Exists

A normal list can hold items.

But if the UI is bound to a list and items are later added or removed, the UI also needs to be informed that the collection itself changed.

That is why the app uses `ObservableCollection<T>` for items such as:

- `Segments`
- `ExtremaReportRows`
- `CoefficientReportRows`
- `CursorReadoutRows`

The key concept is:

- `INotifyPropertyChanged` helps when a property value changes;
- `ObservableCollection<T>` helps when the contents of a collection change.

Those are two related but distinct channels of reactivity.

The Constructor As The Birth Moment

The constructor of `MainWindow` is one of the most educational places in the app.

In simplified terms, it does this:

1. `InitializeComponent();`
2. `DataContext = this;`
3. collects the plot controls into lists;
4. subscribes to collection changes;
5. creates the initial segment;
6. resets the plot after load.

This is almost the birth sequence of the window.

The most important line to recognize first is:

```
InitializeComponent();
```

That call connects the C# partial class with the XAML-defined visual tree. Without it, the controls declared in XAML would not be constructed and attached properly.

What Happens When The User Clicks `Compute`

This is the first full interaction loop you should understand.

In broad form:

1. the user clicks the `Compute` button;
2. XAML routes that click to `Compute_Click`;
3. `Compute_Click` calls `_trajectoryService.Compute(Segments)`;
4. the result object contains series, report rows, delegates for evaluation, and duration;
5. the window replaces the report rows;

6. the window builds the plots;
7. the window updates the cursor readout;
8. the status message is updated;
9. if something fails, an exception is caught and a message box is shown.

Notice the educational richness of this one path:

- event handling;
- collection replacement;
- service invocation;
- exception handling;
- UI refresh;
- state update.

This is why I chose this vertical slice as the first lesson.

What Is Good In This Design For Learning

These choices are especially good for a first teaching phase:

- the interaction flow is visible;
- the bindings are concrete;
- the state is easy to locate;
- the event handlers are explicit;
- the app is doing something meaningful, not artificial.

This means the code has educational density: many important ideas are present in a relatively small space.

What Is Provisional In This Design

This app is good for learning, but we must not confuse "good for learning" with "final ideal architecture".

At this stage:

- `MainWindow` carries many responsibilities;
- `DataContext = this` is practical, but not the final shape of a more mature MVVM design;
- commands are not yet separated from click handlers;
- the prototype service is a bridge, not the whole architectural destination.

This is not a defect for our first lesson. It is actually useful, because it lets us see the machinery before we learn more refined separations.

Vocabulary To Keep

These words are now part of your active vocabulary:

- WPF
- XAML
- code-behind
- `Window`
- `DataContext`
- binding
- property
- change notification
- `INotifyPropertyChanged`
- `ObservableCollection<T>`
- event handler
- service

You do not yet need perfect mastery. But you should begin to feel that each word corresponds to a function in the living system.

Questions For Yourself

Try to answer these without looking back immediately:

1. Why is `App.xaml` necessary if `MainWindow.xaml` already exists?
2. What does `DataContext = this` allow the XAML to do?
3. Why is a property alone not enough for the UI to react?
4. Why is `ObservableCollection<T>` different from a normal `List<T>` in a WPF app?
5. What is the difference between declaring a button in XAML and handling its click in C#?
6. Why do I call this window a "living object" rather than a static page?

First Exercise Without Writing Code

Describe the current app as a circulation of responsibilities using only these nouns:

- application
- window
- XAML
- code
- property
- collection
- service
- user
- result

If you can do that clearly, then Lesson 1 is already working.

Closing Position

This lesson is the doorway, not the destination.

If you truly understand this lesson, then later we can study C# syntax, binding mechanics, MVVM, and architectural refinement on living ground instead of on empty abstractions.

That is why we began here.